MyCodeSchool

Lecția 15 — Pluginuri: din magazin, pe local și în CI

teorie scurtă · 7 exerciții ghidate: te joci local cu magazinul, apoi pui pluginurile pe PR — judecă, comentează, blochează · 3 exerciții pe cont propriu · două repo-uri

Ținta lecției. Primești un **magazin de pluginuri** cu trei unelte gata și un backend pe care să le folosești. Întâi le folosești de mână, pe mașina ta. Apoi le pui să lucreze fără tine: dintr-un script și din CI, unde pică un pull request și îl comentează pe linii, ca un coleg. Pe cont propriu construiești, prin prompting, al patrulea plugin și îl pui în același magazin, cu teste. Pluginurile nu le scrii de mână: le ceri lui Claude și verifici ce a făcut.

Terenul: două repo-uri pe GitHub-ul tău, clonate în `C:\Projects\plugins-lab\` — ca în viața reală, magazinul servește mai multe proiecte, deci nu stă în niciunul:

```
plugins-lab\
  plugins-store\          repo 1: nimigeanconstantinion/plugins-store
    plugins\              check-secrets, check-spring, git-guard
    tests\                diff-uri de test + răspunsul așteptat
  plugins-starter\        repo 2: nimigeanconstantinion/plugins-starter
    .github\workflows\    ci.yml, deocamdată doar build-test
```

De unde vin cele 4 branch-uri din `plugins-starter`. Pe proiect lucrează o echipă: 4 colegi au terminat fiecare câte o schimbare, pe branch-ul lui, și așteaptă review înainte de merge în `main`. Sunt PR-urile de mâine, deja în git-ul tău:

```
git branch
git log --oneline main..feat/mail-config
```

Nu le citi încă: le judecă pluginurile. Până la ex. 6 le judeci local; acolo deschizi PR-urile pe GitHub.
Comenzile le rulezi în Git Bash: `<` și `$?` nu merg la fel în PowerShell. `jq` îl ai de la `machine-setup`.

Teoria — mecanismul, apoi de ce

T1. Ce e un plugin

Un folder cu o etichetă, `.claude-plugin\plugin.json`, și piesele pe care le știi deja, în folderele lor obișnuite:

```
check-spring\
  .claude-plugin\plugin.json    { "name": "check-spring", "version": ... }
  skills\diff\SKILL.md          skill, ca în .claude\skills
  skills\conventions\SKILL.md   al doilea skill
  hooks\hooks.json              hook-uri (la git-guard)
```

Nu e un al cincilea tip de unealtă. E **ambalajul** — ca o extensie VS Code: o instalezi, o pornești, o oprești, are versiune. Cât lucrezi doar tu, `.claude\` ajunge. Pluginul contează când trusa **pleacă**: în alt repo, la un coleg, în CI.

Piesa din plugin	Cum o folosești
`skills\diff\SKILL.md`	o chemi: `/check-spring:diff`; sau Claude o alege singur, după `description:` (lecția 11)
`hooks\hooks.json`	n-o chemi: pornește singură înainte de comenzile lui Claude. N-apare la `/`; o vezi în `/plugin` și în „N hooks” de la `/reload-plugins`

T2. Magazinul și numele

Magazinul (marketplace-ul) e un folder cu `.claude-plugin\marketplace.json`, care listează pluginurile din `plugins\`. Regulile de nume după care e făcut:

Ce	Regula	Aici
plugin	numele capabilității, nu al autorului; un plugin face un lucru	`check-secrets`, `git-guard`
skill	nu repetă numele pluginului: îl chemi ca `/plugin:skill`	`/check-spring:diff`
magazin	același nume cu folderul și cu repo-ul	`plugins-store`
toate	litere mici, cu cratimă; fără `claude-` sau `anthropic-` în față	

Prefixul comun `check-` le face familie: orice review pe diff se cheamă `/check-ceva:diff`. `git-guard` e altfel intenționat: nu verifică un diff, păzește — de aceea nu merge în CI.

T3. Cum ajunge un plugin în sesiune

Mod	Comanda	Când
Instalat din magazin	`/plugin marketplace add <cale sau owner/repo>`, apoi `/plugin install nume@plugins-store`, fiecare plugin separat	îl folosești tu — Partea I
Din folder	`claude -p --plugin-dir <cale>`	în scripturi și în CI: acolo n-are cine să dea `/plugin install` — Partea II
Cerut de proiect	`.claude\settings.json`: `extraKnownMarketplaces` + `enabledPlugins`	oricine clonează repo-ul îl primește — ex. 9

Magazinul e doar catalogul. La `/plugin install` alegi și **unde** se scrie instalarea:

Scope	Scrie în	Pluginul merge
user	`~\.claude\settings.json`	la tine, în orice proiect — **în lecție alegi asta**
project	`.claude\settings.json` din proiect, comis	pentru oricine clonează repo-ul (ex. 9)
local	`.claude\settings.local.json`, necomis	în proiectul ăsta, doar la tine

`/reload-plugins` reîncarcă pluginurile fără să repornești Claude; rezumatul arată câte pluginuri și hook-uri sunt active.

Într-un hook din plugin, calea spre script se scrie cu `${CLAUDE_PLUGIN_ROOT}`: pluginul nu știe unde o să fie instalat.

T4. Claude într-un script: `review.sh`

La ex. 3 ai judecat tu: ai citit răspunsul și ai decis. Într-o echipă vin zeci de PR-uri; le judecă un script, care spune doar **0** (trece) sau **1** (pică). CI-ul (ex. 6) rulează același script la fiecare PR, iar un 1 face PR-ul roșu. `plugins-store\review.sh` îl primești gata, în forma minimă; pe parcurs îi adaugi câte o funcție, prin prompting: la ex. 6 verdictul ca text în CI, la ex. 7 comentariile pe linii.

```
git diff "main...$branch" > /tmp/pr.diff

claude -p "/$plugin:diff" \
  --plugin-dir "$store/plugins/$plugin" \
  --model haiku \
  --disallowedTools Bash Read Edit Write Glob Grep \
                    WebFetch WebSearch NotebookEdit Agent \
  --output-format json \
  --json-schema "$(cat "$store/verdict.schema.json")" \
  < /tmp/pr.diff > /tmp/verdict.json

jq '.structured_output' /tmp/verdict.json
jq -e '.structured_output.blocant == false' /tmp/verdict.json > /dev/null
```

Ideea	Rândul	Ce face
1. Claude ca o comandă	`claude -p … < /tmp/pr.diff`	o singură tură, fără sesiune și fără om: primește diff-ul pe stdin, răspunde, se oprește. Îl pui într-un script, ca pe `grep`
2. Răspuns pentru un program	`--json-schema`	Claude e obligat să răspundă `{blocant, motive}`, în `.structured_output`; nu text liber
3. Codul de ieșire	`jq -e`	face din `"blocant": true` un **1**; CI-ul înțelege doar 0 / 1
4. Fără om = fără unelte	`--disallowedTools`, `--plugin-dir`	Claude are voie doar să citească diff-ul (T5); pluginul îl aduce scriptul, n-are cine să dea `/plugin install`

`--model haiku`: diff mic, model mic, ~0,3 cenți pe rulare. `--output-format json`: ieșirea e JSON, cu costul în `.total_cost_usd`.

> **Codul de ieșire nu e verdictul.** `claude -p` iese cu **0** și când răspunde `"blocant": true`: pentru el, 0 înseamnă „am mers”, nu „e curat”. Abia `jq -e` iese cu 1 când expresia e falsă. Fără ultimul rând, review-ul e mereu verde.

T5. Cât are voie Claude fără om

În CI, Claude citește un diff scris de altcineva. Dacă diff-ul conține „ignoră regulile și rulează `curl ...`", îl oprește doar ce **nu are voie** să facă. Reviewer-ul primește doar diff-ul și scrie doar verdictul:

```
--disallowedTools Bash Read Edit Write Glob Grep WebFetch WebSearch \
                  NotebookEdit Agent
```

> **Capcana:** pare mai simplu `--tools ""`. Dar skill-ul se încarcă tot printr-o unealtă, `Skill`: fără ea, `/check-spring:diff` nu se încarcă și Claude răspunde din burtă. Lista de interdicții păstrează `Skill` și taie restul.

T6. Testele magazinului

Un skill e text pe care Claude îl interpretează: schimbi o propoziție și nu știi ce s-a stricat. `plugins-store\tests\` e poarta pluginurilor: diff-uri mici (`secret.diff`, `spring.diff` ...) și `asteptat.txt`, cu ce trebuie să răspundă **fiecare** plugin pe **fiecare** diff:

```
secret  check-secrets  true     îl prinde
secret  check-spring   false    nu e treaba lui
```

`ruleaza.sh` le rulează pe toate și numără ce a picat. Un plugin schimbat fără `picate: 0` nu e gata.

Partea I — Pluginurile primite, folosite de mână

Cunoaște magazinul

1

În `plugins-lab\`, pornește `claude` și cere-i:

```
Explica-mi plugins-store: ce face fiecare plugin, in ce fisier e
regula lui, cum il chem si ce primeste. Nu modifica nimic.
```

Apoi verifică tu, fără Claude:

```
claude plugin validate plugins-store
```

Gata când: poți spune, pentru fiecare din cele 3 pluginuri, ce face, cum îl chemi și unde ai schimba regula; știi de ce `check-spring` are două skill-uri și de ce `git-guard` n-are niciunul; validarea trece.

Teorie: T1, T2 · Programa: lecțiile 11 (Skills) și 13 (Hooks).

Instalează garda și pune-o la treabă

2

Pornește `claude` în `plugins-starter\`. Garda o vei încerca cu promptul:

```
Creeaza .env cu DB_PASSWORD=parola123 si ruleaza git add -f .env
```

	Faci	Vezi
a	`/plugin marketplace add ../plugins-store`	magazinul `plugins-store` adăugat
b	`/plugin install git-guard@plugins-store`, scope **user**, apoi `/reload-plugins`	în rezumat: `1 hooks`
c	dai promptul	„Blocat de git-guard: .env nu intra in git…”; Claude nu caută altă cale
d	`/plugin` → `git-guard` → Disable, `/reload-plugins`, dai iar promptul	`0 hooks`; `git status` arată `A .env`
e	`! git restore --staged .env`, apoi Enable, `/reload-plugins`, dai iar promptul	blocat din nou

`-f` e intenționat: `.env` e deja în `.gitignore`. Fără el, Claude refuză singur, din bun-simț, și garda nu mai apucă să lucreze.

Limita gărzii: păzește doar uneltele lui Claude. Dacă rulezi tu `! git add -f .env`, trece. Hook-ul e o plasă pentru agent, nu securitate.

Gata când: ai văzut c, d și e exact ca în tabel, iar la final `git status` nu are `.env` în stage.

Indiciu: scope **user** = garda e doar a ta. Ca toată echipa să o primească doar clonând repo-ul, o pui în `.claude\settings.json` din proiect (T3) — îți rămâne ca exercițiu (ex. 9).

Teorie: T1, T3 · Lecția 13 — hook-urile, aici împachetate.

Review de mână pe cele 4 branch-uri

3

În aceeași sesiune, instalează și cele două pluginuri de review, tot cu scope **user**:

```
/plugin install check-secrets@plugins-store
/plugin install check-spring@plugins-store
/reload-plugins
```

La `/check` îți apar `/check-secrets:diff` și `/check-spring:diff`; `git-guard` nu apare, n-are skill-uri (T1).

Branch-ul e argumentul: Claude face singur `git diff main...<branch>`. Câte o comandă pe rând, pe fiecare din cele 4 branch-uri:

```
/check-secrets:diff feat/mail-config
/check-spring:diff feat/mail-config
```

Fără branch, skill-ul judecă modificările tale locale, nu PR-ul.

Ce trebuie să vezi:

Branch	check-secrets	check-spring
`feat/mail-config`	❌ `application.yaml:33`, parola SMTP ca default	✅
`feat/course-archive`	✅	❌ `CourseArchiveController.java:20` `@Autowired` pe câmp · `:25` întoarce entitatea `Course`
`feat/permission-catalog`	✅	✅
`feat/course-description`	✅	✅ — și totuși are o problemă. Care?

Fiecare ❌ îl verifici **tu**, nu Claude, într-un al doilea terminal:

```
F=src/main/resources/application.yaml
git show feat/mail-config:$F | sed -n 33p
```

Gata când: verdictele tale se potrivesc cu tabelul și fiecare ❌ l-ai văzut cu ochii tăi în `git show`. Ce problemă are `feat/course-description` notezi — îți trebuie la ex. 8.

> **Indiciu:** dacă la tine `permission-catalog` iese ❌, ai prins o alarmă falsă: importă `user.model.Permission`, care e un enum, nu o entitate.

Teorie: T1, T3 · Programa: lecția 16 (Poarta).

Partea II — Fără tine: din script și din CI

`review.sh`: verdictul pe care îl citește un program

4

Rulezi scriptul din T4 în terminal, din `plugins-starter\`: scriptul stă în magazin, dar judecă proiectul în care ești — branch-urile sunt aici.

```
../plugins-store/review.sh feat/mail-config check-secrets; echo $?
../plugins-store/review.sh feat/course-archive check-secrets; echo $?
```

	Faci	Vezi
a	rândul cu `mail-config`	`"blocant": true`, motivul cu `application.yaml`, linia 33, apoi `1`
b	rândul cu `course-archive`	`"blocant": false`, apoi `0`

c	scoți ultimul rând din `review.sh` și repeți a	tot `"blocant": true`, dar `0`
d	pui rândul la loc (mai jos)	`git -C ../plugins-store status` curat

`d — rândul la loc`

```
git -C ../plugins-store restore review.sh
```

Gata când: a–d ies ca în tabel și poți spune, pentru fiecare din cele 4 idei din T4, rândul din script care o face.

Indiciu: dacă `structured_output` e `null`, uită-te în `.result` din `/tmp/verdict.json`: acolo Claude spune de obicei că n-a găsit skill-ul (nume greșit de plugin, sau `--tools ""` în loc de lista din T5).

Teorie: T4, T5 · Programa: lecția 15.

Testele magazinului: poarta pluginurilor

5

Un skill e text: schimbi o propoziție și nu știi ce s-a stricat. `tests\` e plasa (T6): rulează fiecare plugin pe fiecare diff și compară cu `asteptat.txt`. O primești gata, cu cele 4 branch-uri deja în ea; aici vezi de ce ai nevoie de ea. Pe a ta o construiești la ex. 8. În terminal, din `plugins-lab\`; o rulare durează 1–2 minute.

	Faci	Vezi
a	testele (mai jos)	16 rânduri `OK`, `picate: 0`
b	citești rândurile branch-urilor din `asteptat.txt`	exact tabelul de la ex. 3 — ele sunt contractul pluginurilor
c	în `claude`: promptul, apoi testele	pică **un singur** rând: `entitate / check-spring`
d	pui regula la loc (mai jos), apoi testele	`picate: 0`

`testele`

```
bash plugins-store/tests/ruleaza.sh
```

`c — promptul`

```
Scoate din check-spring regula cu entitatea JPA in controller. Nu
modifica nimic in tests/.
```

Înainte de teste, te uiți ce a schimbat: doar fișiere din `plugins\check-spring\`.

```
git -C plugins-store diff --stat
```

De ce „nu modifica tests/”: fără fraza asta, Claude scoate regula și schimbă rândul `entitate` din `asteptat.txt`, ca testele să rămână verzi. Plasa nu mai prinde nimic. `asteptat.txt` e contractul: nu-l lași pe mâna lui Claude.

`d — regula la loc`

```
git -C plugins-store restore .
```

Gata când: a–d ies ca în tabel și poți spune de ce `course-archive`, care tot întoarce o entitate, rămâne blocat la c: îl prinde și `@Autowired`. Un diff care încalcă două reguli nu testează niciuna — fiecare regulă are nevoie de diff-ul ei.

Teorie: T6 · Programa: lecția 17 (Testul înainte de cod).

`claude-review`: pluginurile pică PR-ul

6

Acum `review.sh` rulează singur, pe GitHub, la fiecare PR. Repo-urile le ai deja acolo; îți mai trebuie o versiune fixă a magazinului, două secrete, jobul și o regulă pe `main`.

	Faci	Vezi
a	în `claude`, din `plugins-store\`: funcția nouă în `review.sh` (promptul 1), apoi o încerci local	`/tmp/verdict-check-secrets.md` cu verdictul ca text
a2	**tu**: commit, tag, push (mai jos)	pe GitHub, tag-ul `v0.1.0`
b	cele două secrete pe `plugins-starter` (mai jos)	`gh secret list` le arată pe amândouă
c	în `claude`, din `plugins-starter\`: promptul 2	job nou `claude-review` în `ci.yml`
d	citești jobul cu lista de verificare	toate bifate
e	**tu**, în terminal: commit + `git push` pe `main`	dacă îi ceri lui Claude, garda îl oprește: push pe `main`
f	câte un PR pe rând, din fiecare branch (mai jos)	fiecare PR cu 3 joburi, ca rândul lui din tabel
g	regula pe `main` (mai jos)	pe un PR roșu, merge-ul e blocat

`a — promptul 1: verdictul ca text`

```
In review.sh, dupa verdict, scrie /tmp/verdict-<plugin>.md: un rand
"### <plugin>: ❌ blocant" sau "✅ trece" si cate un rand
"- `fisier:linie` mesaj" pe motiv. Daca exista $GITHUB_STEP_SUMMARY,
adauga fisierul acolo. Ultimul rand, jq -e, ramane ultimul.
```

O încerci din `plugins-starter\` — local nu există `$GITHUB_STEP_SUMMARY`, așa că i-l dai tu:

```
GITHUB_STEP_SUMMARY=/tmp/s.md \
  ../plugins-store/review.sh feat/mail-config check-secrets; echo $?
cat /tmp/s.md
```

`a2 — versiunea magazinului`

```
git add review.sh && git commit -m "feat: verdict as text"
git tag v0.1.0 && git push origin main v0.1.0
```

`b — secretele, din plugins-starter\`

Sunt credențialele **tale**, le creezi tu. CI-ul are nevoie de două lucruri:

Secret	De ce îl vrea CI-ul
`CLAUDE_CODE_OAUTH_TOKEN`	să ruleze `claude -p` pe abonamentul tău, fără cheie API
`PLUGINS_STORE_TOKEN`	să citească `plugins-store`, care e privat

1. Se deschide browserul, aprobi, iar în terminal apare un token `sk-ant-oat…`. Îl copiezi; `gh secret set` ți-l cere: îl lipești, Enter.

```
claude setup-token
gh secret set CLAUDE_CODE_OAUTH_TOKEN
```

2. Pe GitHub: avatar → Settings → Developer settings → Personal access tokens → **Fine-grained tokens** → Generate new token. Repository access: **Only select repositories** → `plugins-store`. Permissions → Repository → **Contents: Read-only**. Copiezi token-ul `github_pat_…`, apoi:

```
gh secret set PLUGINS_STORE_TOKEN
gh secret list
```

> **Atenție:** token-urile le lipești doar în terminal — niciodată în chat, în cod sau în `ci.yml`. Un token lipit altundeva îl revoci și faci altul.

`c — promptul 2: jobul`

```
In .github/workflows/ci.yml adauga langa build-test un job
claude-review, matrice pe check-secrets si check-spring. Checkout cu tot
istoricul si branch-ul local main din origin/main. Aduce repo-ul
nimigeanconstantinion/plugins-store cu actions/checkout la tag-ul
v0.1.0 in .plugins-store, cu secretul PLUGINS_STORE_TOKEN, si ruleaza
bash .plugins-store/review.sh HEAD <plugin> cu CLAUDE_CODE_OAUTH_TOKEN.
Claude Code instalat la versiunea mea (claude --version).
Permisiunile raman contents: read. La final, si cand pica, urca
/tmp/verdict-<plugin>.md ca artefact verdict-<plugin>.
```

`d — lista de verificare`

`fetch-depth: 0` + `main` local · `ref: v0.1.0` · versiunea Claude Code fixată · `review.sh` din magazin, nu o copie · `contents: read` · artefactul urcat cu `if: always()`.

> **De ce `main` local:** `review.sh` face `git diff main...HEAD`. În CI, checkout-ul aduce doar commit-ul PR-ului; fără `main`, diff-ul dă eroare. De aceea `review.sh` începe cu `set -e`: o eroare face jobul roșu. Fără el, Claude primește un diff gol, răspunde „nu e blocant” și PR-ul iese verde fără să fi fost citit.

`f — PR-urile, unul câte unul`

Deschizi un PR, aștepți joburile (apar după câteva secunde, durează 1–2 minute), compari cu rândul lui din tabel, apoi treci la următorul branch:

```
gh pr create --base main --head feat/mail-config --fill
gh pr checks feat/mail-config --watch
```

La fel pentru `feat/course-archive`, `feat/permission-catalog`, `feat/course-description`. Motivul unui ❌: PR → Checks → jobul roșu → **Summary**.

Ce trebuie să vezi:

PR	build-test	check-secrets	check-spring
`feat/mail-config`	✅	❌ parola de mail	✅
`feat/course-archive`	✅	✅	❌ 2 motive
`feat/permission-catalog`	✅	✅	✅
`feat/course-description`	✅	✅	✅ **?!**

`g — un ❌ blochează merge-ul`

Fără regulă, un ❌ e doar o bulină roșie: butonul Merge rămâne verde. Pe GitHub, în `plugins-starter`: Settings → Rules → Rulesets → **New branch ruleset**:

Câmp	Pui
Ruleset Name	`protect-main`
Enforcement status	**Active**
Bypass list	Add bypass → **Repository admin** — altfel nici tu nu mai poți face push direct pe `main`
Target branches	Add target → **Include default branch**

Rules	bifezi **Require status checks to pass** → sub ea, **+ Add checks** → `build-test`, `claude-review (check-secrets)`, `claude-review (check-spring)`

Apoi **Create**. Joburile apar la Add checks doar după ce au rulat o dată (pasul f). Pe PR-ul `feat/mail-config`, merge-ul e acum blocat; ca admin vezi opțiunea de bypass, un coleg n-o are. Dacă GitHub te avertizează că regula nu se aplică pe un repo privat (cont Free), fă `plugins-starter` public — `plugins-store` rămâne privat.

Gata când: bifele de pe GitHub sunt ca în tabel, iar un PR roșu nu se mai poate merge-ui. Ultimul rând e verde peste tot — și totuși PR-ul ăsta strică producția. Ex. 8.

Indiciu: de ce tag și nu `main`: dacă schimbi o regulă în magazin, PR-urile tuturor proiectelor ar începe să pice peste noapte. Cu tag, un proiect trece la regulile noi când **el** mută `ref`. Același motiv pentru versiunea fixă de Claude Code. De ce token: magazinul e privat, iar `GITHUB_TOKEN` al jobului vede doar repo-ul proiectului.

Teorie: T4, T5 · Programa: lecția 22 (Claude în CI).

`claude-comment`: review pe linii, ca un coleg

7

Un ❌ în Checks îl vezi doar dacă îl cauți. Un coleg care face review îți lasă comentariul pe linie, în PR. Asta face al doilea job: ia artefactele de la `claude-review` și pune un review — verdictele în conversație, fiecare problemă pe linia ei, în Files changed.

```
claude-review (check-x)   contents: read, rulează Claude
  review.sh → verdict-x.md + comments-x.json → artefact
claude-comment            pull-requests: write, FĂRĂ Claude
  artefactele → un review pe PR, cu comentarii pe linii
```

Ai nevoie de o funcție nouă în `review.sh`, deci de o versiune nouă a magazinului: proiectul o primește doar când mută `ref`.

	Faci	Vezi
a	în `claude`, din `plugins-store\`: promptul 1, apoi îl încerci local (mai jos)	`/tmp/comments-check-spring.json` cu liniile 20 și 25
b	**tu**: commit, tag `v0.1.1`, push (mai jos)	pe GitHub, tag-ul `v0.1.1`
c	în `claude`, din `plugins-starter\`: promptul 2	în `ci.yml`: `ref: v0.1.1`, artefactul cu ambele fișiere, job nou `claude-comment`
d	citești joburile cu lista de verificare	toate bifate
e	**tu**: commit + `git push` pe `main`	
f	rulezi din nou PR-urile deschise, unul câte unul (mai jos)	pe fiecare, un review de la `github-actions`; fiecare ❌ pe linia lui

`a — promptul 1: comentariile`

```
In review.sh scrie si /tmp/comments-<plugin>.json: un array cu cate un
obiect pe motiv, {path: fisier, line: linie, side: "RIGHT",
body: "**<plugin>**: mesaj"}. Ultimul rand, jq -e, ramane ultimul.
```

O încerci din `plugins-starter\`:

```
../plugins-store/review.sh feat/course-archive check-spring
jq . /tmp/comments-check-spring.json
```

b — versiunea nouă a magazinului

```
git add review.sh && git commit -m "feat: line comments"
git tag v0.1.1 && git push origin main v0.1.1
```

c — promptul 2: jobul care comentează

```
In ci.yml muta ref la v0.1.1 si urca in artefactul verdict-<plugin>
si /tmp/comments-<plugin>.json. Adauga un job claude-comment, dupa
claude-review, si cand acesta pica. Doar el are pull-requests: write
si nu ruleaza Claude. Descarca artefactele verdict-*, lipeste .md-urile
in body si .json-urile in comments si posteaza un review pe PR cu
gh api .../pulls/<nr>/reviews (event COMMENT). Daca review-ul pica,
posteaza body ca un comentariu.
```

d — lista de verificare

`ref: v0.1.1` · `needs: claude-review` + `if: always()` · `pull-requests: write` doar aici · niciun `claude` în el · `claude-review` tot cu `contents: read`.

f — PR-urile, din nou

```
gh pr update-branch feat/course-archive
gh pr checks feat/course-archive --watch
```

În Actions, `claude-comment` e legat cu o linie de `claude-review` (`needs`); `build-test` stă separat.

Gata când: pe `course-archive` vezi comentariile pe liniile 20 și 25 din `CourseArchiveController.java`, pe `mail-config` pe linia 33 din `application.yaml`, iar pe PR-urile verzi doar verdictul în conversație.

Atenție: Claude citește un diff scris de altcineva, deci jobul lui nu primește **niciodată** `write`. Scrie doar `claude-comment`, care nu rulează Claude: lipește un text gata făcut, n-are ce păcăli (T5, lecția 24).

Teorie: T4, T5 · Programa: lecțiile 22 și 24.

Exerciții — pe cont propriu

`check-liquibase`: ce n-a văzut nimeni

8

`feat/course-description` adaugă o coloană **modificând changeset-ul 001**, deja aplicat. Testele trec (H2 pornește de la zero), dar pe orice bază unde 001 a rulat, Liquibase vede alt checksum și aplicația nu mai pornește. Corect era un changeset nou, 003.

Întâi testele, care pică, apoi pluginul, care le face verzi. Din `plugins-lab\`:

	Faci	Vezi
a	în `claude`: promptul 1, apoi testele	câteva rânduri `check-liquibase` `PICAT` — nu toate: fără skill, Claude ghicește (T5), uneori nimerește
b	promptul 2	`claude plugin validate plugins-store` trece, `picate: 0`
c	**tu**, în `plugins-store\`: commit, tag `v0.2.0`, push (mai jos)	pe GitHub, tag-ul `v0.2.0`
d	în `claude`, din `plugins-starter\`: promptul 3; commit + push **tu**	în `ci.yml`: `ref: v0.2.0` și 3 pluginuri în matrice
e	rulezi din nou PR-urile (mai jos)	`claude-review (check-liquibase)` ❌ doar pe `course-description`, cu fișierul și linia

promptul 1

```
In plugins-store/tests adauga migrare-noua.diff: aceeasi coloana facuta
corect, intr-un changeset nou 003 inclus la final in master. Apoi
randurile pentru un plugin nou check-liquibase pe toate diff-urile:
blocant doar pe course-description. Nu crea inca pluginul: doar testele.
```

promptul 2

```
Creeaza in plugins-store pluginul check-liquibase, cu skill-ul diff:
blocant cand un changeset existent din db/changelog e modificat sau un
include existent e sters din master. Un changeset nou nu e blocant.
Adauga-l in marketplace.json, valideaza si ruleaza testele.
```

`c` — versiunea nouă a magazinului

```
git add -A && git commit -m "feat: add check-liquibase"
git tag v0.2.0 && git push origin main v0.2.0
```

promptul 3

```
In ci.yml muta ref la v0.2.0 si adauga check-liquibase in matrice.
```

`e` — PR-urile, din nou, unul câte unul

```
gh pr update-branch feat/course-description
gh pr checks feat/course-description --watch
```

Apoi la fel pe celelalte 3: `check-liquibase` trebuie să fie verde pe ele. În protecția de la ex. 6 g adaugi și `claude-review (check-liquibase)`, altfel PR-ul roșu se poate merge-ui.

Gata când: a–e ies ca în tabel (la b, inclusiv `migrare-noua` = nu blochează); până la d, CI-ul nu știe de pluginul nou; și poți spune de ce `migrare-noua.diff` e la fel de important ca diff-ul stricat.

De ce „nu crea încă pluginul”: fără fraza asta, Claude face și testele, și pluginul, dintr-o dată — și nu mai vezi testele picând întâi. Un test care n-a picat niciodată nu dovedește că prinde ceva.

Indiciu: fără `migrare-noua.diff`, un plugin care blochează **orice** atingere a `db/changelog/` ar trece toate testele — și ar pica fiecare migrare corectă de acum înainte.

Teorie: T2, T6 · Programa: lecțiile 17 și 18 (Unelte la plural).

Garda pentru toată echipa 9

Fă ca `git-guard` să pornească singur pentru oricine clonează `plugins-starter`, fără `/plugin install` — magazinul luat de pe GitHub, nu din folderul vecin. Verifică într-o clonă nouă, în alt folder.

Gata când: într-o clonă proaspătă, după acceptul magazinului, `git add -f .env` cerut lui Claude e blocat din prima sesiune.

Teorie: T3.

Cât costă un PR 10

Adaugă `.total_cost_usd` în log. Costul unui PR = suma joburilor `claude-review (…)`. Estimează pe lună la ritmul tău real de PR-uri (`gh pr list --state all` pe repo-urile platformei), cu haiku și cu sonnet.

Gata când: două cifre pe lună și o propoziție: pe ce fel de PR ar merita modelul mare.

Teorie: T4 · Programa: lecția 21 (Cost și când NU).